ReactJS Hands-On Lab

Exercises 1 – 5 : Setup, Components, Lifecycle & Styling

Submitted by: Siddhesh Avhad
Java Full Stack Engineer (FSE) Training Program
VIT Bhopal

Exercise 1 — Setting Up the React Environment

What we are doing here

This first lab is basically the "hello world" of React — get the tooling installed, scaffold a project with create-react-app, and get a heading rendering in the browser. Nothing fancy yet, but it is the foundation everything else in this report builds on.

A bit of theory before jumping in

Single Page Application (SPA)

A Single Page Application loads one HTML shell from the server and then handles all further screen changes on the client side using JavaScript, instead of asking the server for a brand-new page every time. Only the data that actually changed is fetched (usually as JSON) and the DOM is updated in place, which is what makes SPAs feel fast and "app-like" rather than like a stack of separate web pages.

SPA vs MPA (Multi Page Application)

- Page reloads: SPA never does a full reload after the first load; MPA reloads the entire page on every navigation.
- Speed after first load: SPA feels snappier because only data/components change; MPA re-renders everything from scratch each time.
- SEO: MPA is generally easier to get indexed out of the box; SPA needs extra work (SSR, pre-rendering) for good SEO.
- Complexity: SPA pushes routing and state management to the client (more JS complexity); MPA keeps most logic server-side.
- Initial load: SPA can have a heavier first load since it pulls the JS bundle; MPA loads are lighter per page but repeat that cost on every click.

Pros and cons of SPAs

On the plus side: fast subsequent navigation, a smoother user experience close to a native app, less redundant data transfer, and a clean separation between the frontend and the backend API. On the downside: the initial bundle can be large, SEO needs extra effort, and browser back/forward + deep-linking need to be handled deliberately by the framework/router.

What React actually is

React is a JavaScript library (not a full framework) from Meta, used to build user interfaces out of small, reusable pieces called components. Instead of manually touching the DOM, we describe what the UI should look like for a given state, and React figures out the most efficient way to update the real DOM to match.

The Virtual DOM

The virtual DOM is a lightweight, in-memory copy of the actual DOM tree. Whenever component state changes, React first builds a new virtual DOM tree, diffs it against the previous one, and then applies only

the minimal set of real DOM updates needed — instead of re-rendering the whole page. This diffing process is what makes React fast even with frequent UI updates.

Key features of React

- JSX — lets you write HTML-like syntax directly inside JavaScript
- Component-based architecture — UI is broken into independent, reusable pieces
- Virtual DOM — efficient, diff-based rendering
- One-way (unidirectional) data binding — data flows from parent to child via props
- Hooks — useState, useEffect etc. let function components manage state and side effects
- Large ecosystem — React Router, Redux/Context API, huge community support

Prerequisites

- Node.js
- NPM (comes bundled with Node.js)
- Visual Studio Code

Implementation Steps

Step 1: Install Node.js and npm from nodejs.org (npm ships with the Node installer, so a single install covers both).

Step 2: create-react-app doesn't need a separate global install anymore — npx pulls it on demand. Open a terminal and run:

```
npx create-react-app myfirstreact
```

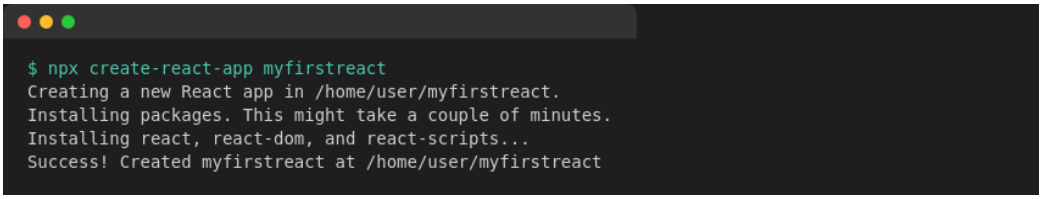


Fig 1.1 — Scaffolding the app with create-react-app

Step 3: Move into the generated project folder:

```
cd myfirstreact
```

Step 4: Open the myfirstreact folder in Visual Studio Code (File → Open Folder).

Step 5: Open src/App.js and clear out the boilerplate JSX that create-react-app generates by default.

Step 6: Replace it with a minimal component that prints the required heading:

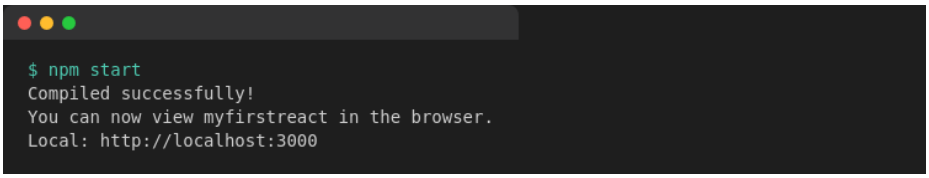
```
function App() {
```

```
  return (
    <div className="App">
      <h1>welcome to the first session of React</h1>
    </div>
  );
}

export default App;
```

Step 7: Start the dev server:

```
npm start
```



Step 8: Open a browser and navigate to localhost:3000 — the heading renders on screen.

Output

Fig 1.2 — myfirstreact running at localhost:3000

Exercise 2 — Building Class Components (Student Management Portal)

What we are doing here

This exercise moves from a single static component to multiple class components that get composed together — Home, About and Contact, each rendered as a separate piece inside App.js. This is the standard pattern for splitting a UI into logical, reusable chunks.

Theory

Components vs. plain JavaScript functions

A component looks like a function but it returns markup (JSX) describing UI, is managed by React's rendering lifecycle, and can hold its own state. A plain JS function just computes and returns a value with no concept of rendering, re-rendering, or lifecycle — React components are specifically wired into React's reconciliation process.

Types of components

- Class components — ES6 classes that extend React.Component and implement a render() method. They were the only way to use local state and lifecycle hooks before React 16.8.
- Function components — plain JavaScript functions that return JSX. Since Hooks arrived, they can do everything class components can, with less boilerplate.

Anatomy of a class component

A class component extends React.Component. If it needs local state, that state is initialized inside a constructor(props) — and the constructor must call super(props) first so React can wire up the base class correctly. Every class component must implement a render() method, which returns the JSX to be displayed; render() is called automatically by React whenever the component needs to be redrawn (initial mount, or after a state/props change).

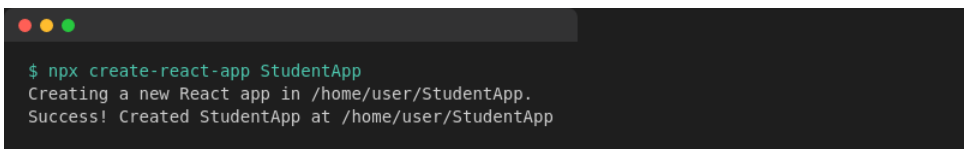
Prerequisites

- Node.js
- NPM
- Visual Studio Code

Implementation Steps

Step 1: Scaffold a new project named StudentApp:

```
npx create-react-app StudentApp
```



Step 2: Inside src, create a new folder called components, and add Home.js in it with a class component that renders the home page message:

```
import React, { Component } from 'react';

class Home extends Component {
  render() {
    return (
      <div className="page">
        <h2>Welcome to the Home page of Student Management Portal</h2>
      </div>
    );
  }
}

export default Home;
```

Step 3: Repeat the same pattern for About.js:

```
import React, { Component } from 'react';

class About extends Component {
  render() {
    return (
      <div className="page">
        <h2>Welcome to the About page of the Student Management Portal</h2>
      </div>
    );
  }
}

export default About;
```

Step 4: And again for Contact.js:

```
import React, { Component } from 'react';

class Contact extends Component {
  render() {
    return (
      <div className="page">
        <h2>Welcome to the Contact page of the Student Management Portal</h2>
      </div>
    );
  }
}

export default Contact;
```

Step 5: Edit App.js to import and render all three components together:

```
import React from 'react';
import Home from './components/Home';
import About from './components/About';
import Contact from './components/Contact';

function App() {
  return (
```

```
    <div className="App">
      <Home />
      <About />
      <Contact />
    </div>
  );
}

export default App;
```

Step 6: Run the app from the project root:

```
npm start
```

Step 7: Open localhost:3000 — all three components render one below the other.

Output

Welcome to the Home page of Student Management Portal

Welcome to the About page of the Student Management Portal

Welcome to the Contact page of the Student Management Portal

Fig 2.1 — Home, About and Contact components rendered together

Exercise 3 — Function Components, Props & Basic Styling

What we are doing here

This one builds a small score calculator as a function component. It takes a student's name, school, total marks and the number of subjects (goal) as props, works out the average, and displays a styled result card.

Theory

Why function components

A function component is just a JavaScript function that accepts props and returns JSX — no class, no constructor, no this keyword to worry about. Since React 16.8 introduced Hooks, function components can do everything class components could (local state via useState, side effects via useEffect), which is why most new React code today is written this way.

Props

Props (short for "properties") are how data is passed from a parent component down into a child component. They are read-only from the child's point of view — a component should never modify its own props, it just consumes whatever the parent hands it and renders accordingly.

Why separate stylesheets

Keeping styles in their own .css file (rather than inline in JSX) keeps components readable and lets the same class names be reused across components. It is imported directly into the component file, and the bundler takes care of including it in the final build.

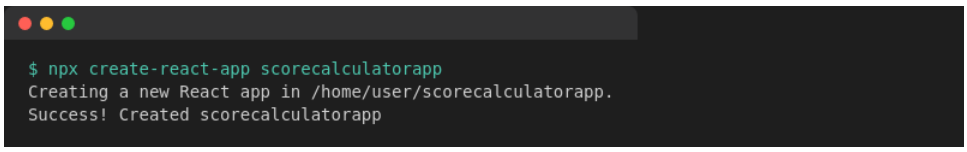
Prerequisites

- Node.js
- NPM
- Visual Studio Code

Implementation Steps

Step 1: Scaffold the project:

```
npx create-react-app scorecalculatorapp
```



Step 2: Create a components folder under src, and add CalculateScore.js — a function component that receives name, school, total and goal as props and computes the average:

```
import React from 'react';
```

```
import '../stylesheets/mystyle.css';

function CalculateScore(props) {
  const { name, school, total, goal } = props;
  const average = (total / goal).toFixed(2);

  return (
    <div className="card">
      <h2>Score Report</h2>
      <p><b>Name:</b> {name}</p>
      <p><b>School:</b> {school}</p>
      <p><b>Total Marks:</b> {total}</p>
      <p><b>Number of Subjects (Goal):</b> {goal}</p>
      <p className="average">Average Score: {average}</p>
    </div>
  );
}

export default CalculateScore;
```

Step 3: Create a stylesheets folder with mystyle.css to give the card some visual polish:

```
.card {
  width: 320px;
  margin: 40px auto;
  padding: 20px 24px;
  border: 1px solid #cfd8dc;
  border-radius: 10px;
  background-color: #ffffff;
  box-shadow: 0 2px 6px rgba(0,0,0,0.08);
  font-family: Arial, sans-serif;
}

.card h2 {
  color: #1565c0;
  border-bottom: 2px solid #1565c0;
  padding-bottom: 8px;
}

.average {
  font-weight: bold;
  color: #2e7d32;
  font-size: 18px;
}
```

Step 4: Edit App.js to invoke CalculateScore with sample data:

```
import React from 'react';
import CalculateScore from './components/CalculateScore';

function App() {
  return (
    <div className="App">
      <CalculateScore
        name="Ananya Sharma"
        school="Delhi Public School"
        total={456}
        goal={5}
      />
```

```
    </div>
  );
}

export default App;
```

Step 5: Run the app:

```
npm start
```

Step 6: Check localhost:3000 — the styled score card should appear with the computed average.

Output

Fig 3.1 — Styled score card computed from props (average = 456 / 5 = 91.20)

Exercise 4 — Component Lifecycle (componentDidMount & componentDidCatch)

What we are doing here

A small blog app that fetches posts from a public API and renders them, used as a vehicle to actually apply two lifecycle hooks: componentDidMount() to trigger the data fetch once the component is on screen, and componentDidCatch() to gracefully catch a rendering error instead of letting the whole app crash.

Theory

Why component lifecycle matters

A class component goes through three broad phases — mounting (being created and inserted into the DOM), updating (re-rendering in response to new props/state) and unmounting (being removed from the DOM). React exposes hook methods at each of these phases so we can run code at exactly the right moment — for example, fetching data right after the component is visible, or cleaning up a subscription right before it disappears.

Sequence of a class component render

- constructor(props) — set up initial state
- render() — React calls this to know what to draw; runs on every re-render
- componentDidMount() — runs once, immediately after the component is first inserted into the DOM — the standard place for API calls, subscriptions, timers
- componentDidUpdate() — runs after subsequent re-renders (not used in this exercise)
- componentWillUnmount() — runs right before the component is removed (not used in this exercise)

componentDidMount()

This hook fires exactly once, right after the component's first render is committed to the DOM. It is the recommended place to start network requests, because the component is guaranteed to already exist — calling setState() here triggers a re-render with the fetched data.

componentDidCatch()

This turns a class component into an "error boundary". If any component further down the tree throws an error during rendering, componentDidCatch(error, info) is invoked instead of the whole app unmounting with a blank white screen. It is commonly paired with getDerivedStateFromError() to swap in fallback UI, though a simple alert or state flag (as used here) is enough to demonstrate the mechanism.

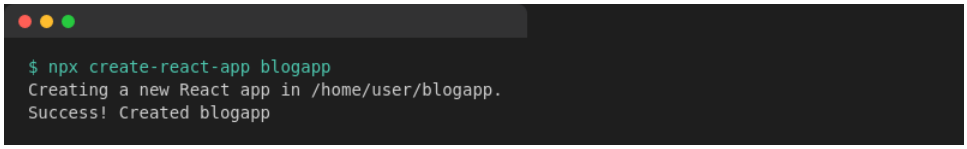
Prerequisites

- Node.js
- NPM
- Visual Studio Code

Implementation Steps

Step 1: Scaffold the project:

```
npx create-react-app blogapp
```



Step 2: Create Post.js — a small presentational component that just displays a title and body it receives as props:

```
import React, { Component } from 'react';

class Post extends Component {
  render() {
    const { title, body } = this.props;
    return (
      <div className="post">
        <h3>{title}</h3>
        <p>{body}</p>
      </div>
    );
  }
}

export default Post;
```

Step 3: Create Posts.js and initialize state with an empty posts array in the constructor:

```
constructor(props) {
  super(props);
  this.state = {
    posts: [],
    hasError: false
  };
}
```

Step 4: Add a loadPosts() method that calls the Fetch API against jsonplaceholder and stores the result in state:

```
loadPosts = () => {
  fetch('https://jsonplaceholder.typicode.com/posts')
    .then(response => response.json())
    .then(data => {
      this.setState({ posts: data.slice(0, 5) });
    })
    .catch(error => {
      console.error('Error fetching posts:', error);
    });
};
```

Step 5: Call loadPosts() from componentDidMount() so the fetch fires as soon as the component is on screen:

```
componentDidMount() {
  this.loadPosts();
}
```

Step 6: Implement render() to map over state.posts and display each one via the Post component:

```
render() {
  if (this.state.hasError) {
    return <h3>Something went wrong.</h3>;
  }
  return (
    <div className="posts">
      <h2>Blog Posts</h2>
      {this.state.posts.map(post => (
        <Post key={post.id} title={post.title} body={post.body} />
      ))}
    </div>
  );
}
```

Step 7: Add componentDidCatch() to catch any rendering error from the Post children and alert instead of crashing:

```
componentDidCatch(error, info) {
  this.setState({ hasError: true });
  alert('Something went wrong while rendering the posts: ' + error.message);
}
```

Step 8: Add the Posts component to App.js:

```
import React from 'react';
import Posts from './Posts';

function App() {
  return (
    <div className="App">
      <Posts />
    </div>
  );
}

export default App;
```

Step 9: Run the app:

```
npm start
```

> **Note:** *The screenshot below was captured in an offline sandboxed environment, so the live jsonplaceholder.typicode.com call was mocked with sample data purely to demonstrate the rendered output. The fetch code itself is unchanged and pulls real posts when run with an active internet connection.*

Output

Fig 4.1 — Posts fetched in componentDidMount() and rendered via the Post component

Exercise 5 — Styling with CSS Modules

What we are doing here

The task here was to style an existing Cohort Details dashboard for a Cognizant Academy team, using a CSS Module instead of a regular global stylesheet, plus a conditional inline-driven color for the cohort status heading.

Theory

Why not just use a regular .css file

A plain, globally-imported CSS file means every class name (.box, .title, etc.) lives in one shared namespace across the whole app — it is very easy for two components to accidentally use the same class name and clobber each other's styles as the project grows.

CSS Modules

A file named *.module.css is treated specially by the build tool: every class name inside it gets automatically scoped to the component that imports it (behind the scenes it is renamed to something like _box_a3f9k at build time). When you write import styles from './CohortDetails.module.css', you get back a plain JS object where each key is your original class name and the value is the generated, collision-free class name — so you apply it as className={styles.box}.

className vs. the style prop

className={styles.box} applies a CSS class (defined in the module file, so it can include hover states, media queries etc.), whereas the style={{...}} prop applies inline styles directly as a JS object — useful for values that are computed at runtime, but it does not support pseudo-classes or media queries.

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Implementation Steps

Step 1: Unzip the provided starter app, open a terminal in that folder and restore the node packages:

```
npm install
```

Step 2: Open the project in VS Code and create CohortDetails.module.css.

Step 3: Define the .box class exactly as specified — 300px wide, inline-block, 10px margin all round, 10px top/bottom padding, 20px left/right padding, a 1px solid black border and a 10px border radius:

```
.box {
  width: 300px;
  display: inline-block;
  margin: 10px;
  padding: 10px 20px;
  border: 1px solid black;
  border-radius: 10px;
}
```

Step 4: Add a tag-selector rule for <dt> elements, setting font-weight to 500:

```
.box dt {
  font-weight: 500;
}
```

Step 5: Open the CohortDetails component and import the CSS Module:

```
import styles from './CohortDetails.module.css';
```

Step 6: Apply the box class to the container div:

```
<div className={styles.box}>
```

Step 7: Define the conditional heading color — green while a cohort's status is "ongoing", blue for every other status — and wire the whole component together:

```
function CohortDetails(props) {
  const { name, trainer, status, startDate } = props;
  const statusClass = status === 'ongoing' ? styles.ongoing : styles.completed;

  return (
    <div className={styles.box}>
      <h3 className={statusClass}>{name}</h3>
      <dl>
        <dt>Trainer</dt><dd>{trainer}</dd>
        <dt>Status</dt><dd>{status}</dd>
        <dt>Start Date</dt><dd>{startDate}</dd>
      </dl>
    </div>
  );
}

.ongoing { color: green; }
.completed { color: blue; }
```

Step 8: Run the app and confirm the cards render with the box styling, and that ongoing cohorts show a green heading while completed ones show blue:

```
npm start
```

```
$ npm start
Compiled successfully!
Local: http://localhost:3000
```

Output

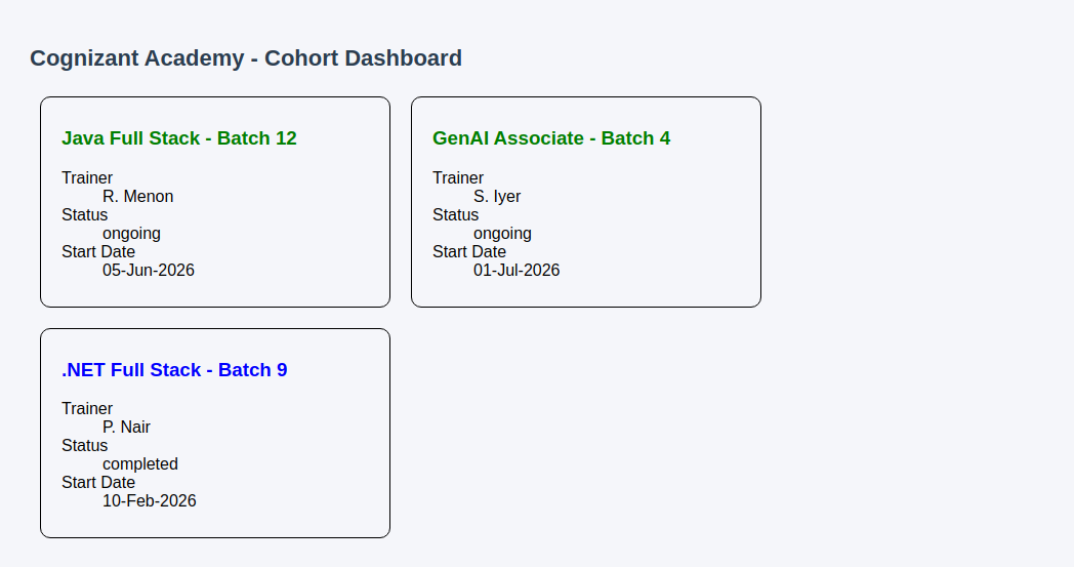


Fig 5.1 — Cohort dashboard styled with CSS Modules; green heading = ongoing, blue = completed